

Algorithm for file updates in Python

Project description

I work as a security professional at a healthcare company and am responsible for managing an allow list of IP addresses that controls access to restricted content containing patient records. As part of my role, I developed a Python algorithm that automatically checks the allow list against a remove list and updates the file by removing any IP addresses that should no longer have access. This helps ensure that only authorized employees can access sensitive patient data.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

To open the file, I assigned the filename "allow_list.txt" to the variable import_file. I then used a with statement combined with the open() function to open the file in read mode ("r"). The with keyword ensures the file is properly closed after use, and the as file syntax assigns the opened file to the variable file for use inside the statement.

Read the file contents

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Inside the with statement, I used the `.read()` method on the file variable to read the entire contents of the file and store them as a string in the variable `ip_addresses`. This converts the file contents into a format that can be worked with in Python.

Convert the string into a list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)
```

I used the `.split()` method on the `ip_addresses` string to convert it into a list. By default, `.split()` separates the string at whitespace characters, which in this case splits each IP address into its own individual list element. This is necessary because removing individual IP addresses requires them to be in list format.

Iterate through the remove list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Display `element` in every iteration
    print(element)
```

I set up a for loop using for element in ip_addresses: to iterate through each IP address in the list. The loop variable element represents each individual IP address as the loop moves through the list one item at a time.

Remove IP addresses that are on the remove list

```
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Loop through `ip_addresses`
for element in ip_addresses:
    # If current element is in `remove_list`,
    if element in remove_list:
        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

print(ip_addresses)
```

Inside the for loop, I added a conditional statement if element in remove_list: to check whether each IP address in the allow list also appears in the remove list. If the condition is true, I applied the .remove() method to delete that IP address from the ip_addresses list. This approach works correctly because there are no duplicate IP addresses in the allow list, so .remove() will only target one instance of each address.

Update the file with the revised list of IP addresses

```
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

for element in ip_addresses:

    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
```

After removing the necessary IP addresses, I converted the updated `ip_addresses` list back into a string using the `.join()` method with a space separator. I then used a second `with` statement and opened the file in write mode ("w") to overwrite the original file with the updated list using the `.write()` method.

Summary

In this project I developed a Python algorithm to automate the process of updating an IP address allow list for a healthcare company. The algorithm opens the allow list file using a `with` statement and `open()` function, reads its contents with `.read()`, and converts the string into a list using `.split()`. It then iterates through the remove list using a `for` loop and removes any matching IP addresses using a conditional statement and the `.remove()` method. Finally the updated list is converted back to a string with `.join()` and written back to the file using `.write()`. This algorithm makes sure that access to restricted patient data is automatically kept up to date.